

Parallel Skeleton Selection: Scenario-Based Guide

A major part of PDC exams is not just writing code, but **justifying your architectural design**. Professors love giving you a real-world algorithm (like K-Nearest Neighbors, PageRank, or Image Processing) and asking: "Which Parallel Skeleton is best here, and why?"

This guide breaks down exactly **when to use which skeleton**, followed by detailed mock exam questions focused on design justification.

1. The Core Decision Tree

When looking at an algorithm, ask yourself these three questions:

1. **Are the tasks completely independent of each other (no shared data)?** -> (Map or Farm)
 2. **Do the tasks take varying amounts of time?** -> (Farm over Map)
 3. **Do the tasks need to be combined into a single answer?** -> (Reduce)
 4. **Is there a strict data dependency?** -> (Wavefront, Scan, or Pipeline)
-

2. Deep Dive: When to use what

 Farm (Master-Worker) vs.  Map

Both handle embarrassingly parallel problems (independent tasks). **The crucial difference is Load Balancing.**

- **Map Limitations:** Map statically divides the data (N / P). If you have 4 threads, thread 1 gets the first 25% of the array. If the first 25% takes 10 hours to compute, and the rest takes 1 second, Threads 2, 3, and 4 will sit idle for 10 hours.
- **Farm Strengths:** Farm uses a queue. Workers grab 1 task at a time. If Thread 1 gets a 10-hour task, Threads 2, 3, and 4 will simply process all remaining tasks in the queue. No one sits idle.
- **The Professor's Example (Unsupervised KNN):** In K-Nearest Neighbors, calculating the distance to every point is completely independent. However, depending on the data density or the search tree (like a KD-Tree), finding the nearest neighbors for Point A might take longer than finding them for Point B. A Farm skeleton uses a queue to dynamically balance this workload across workers perfectly, ensuring maximum CPU utilization. Farm is also superior when tasks arrive as a continuous *Stream* rather than a fixed array.

 Reduce vs.  Scan

Both deal with aggregating data.

- **Reduce:** You only care about the *Final Answer* (e.g., Total Sum, Absolute Maximum).
- **Scan (Prefix Sum):** You care about the *Intermediate Steps* (e.g., Running Total, Cumulative probabilities). If you need to output an array of the same size as the input, you need Scan.

 Wavefront vs.  Pipeline

Both deal with strict data dependencies.

- **Wavefront (Grid Dependency):** Used when data is in a 2D/3D grid, and cell (i,j) mathematically requires the answers of its immediate neighbors $(i-1, j)$ and $(i, j-1)$ to be calculated first. You parallelize by computing across the *diagonals*. (Examples: Dynamic Programming, LCS, Heat/Fluid simulations).
- **Pipeline (Stage Dependency):** Used when a single item of data must pass through distinct processing stages: $A \rightarrow B \rightarrow C$. You parallelize by having Thread 1 do Stage A on item #5, while Thread 2 does Stage B on item #4. (Examples: Video Encoding, CPU instruction pipelines, compiling code).

3. Real-World Exam Scenarios

Scenario 1: Raytracing a 3D Scene

The Algorithm: A 3D renderer shoots a light ray through every single pixel on a 1080p screen. Some pixels hit the sky (which takes 1 millisecond to calculate). Some pixels hit a glass mirror reflecting onto a shiny car (which takes 500 milliseconds to calculate the complex light bounces). **The Question:** Which skeleton should you use to render the image fastest across 8 cores? Justify your choice over its primary alternative. **The**

Answer:

- **Skeleton:** Farm (Master-Worker)
- **Justification:** Raytracing pixels is an embarrassingly parallel problem (Pixel A does not depend on Pixel B). The primary alternative is a Map skeleton. However, Map would statically assign block regions (e.g., the top 1/8th of the screen) to threads. Since the sky takes no time to render, threads assigned to the sky would finish instantly and sit idle, while threads assigned to the complex glass object would bottleneck the entire program. A Farm skeleton with a task queue provides **dynamic load balancing**, ensuring all 8 cores stay 100% busy until the very last pixel is done.
- **Code Injection (Python Farm):**

```
import multiprocessing as mp

def render_pixel(queue, results):
    while True:
        task = queue.get()
        if task is None: break # Poison pill

        x, y = task
        color = calculate_light_bounces(x, y) # Fast for sky, slow for glass
        results.put((x, y, color))

# Master setup
task_q = mp.Queue()
res_q = mp.Queue()

# Spawn 8 workers
workers = [mp.Process(target=render_pixel, args=(task_q, res_q)) for _ in range(8)]
for w in workers: w.start()
```

Scenario 2: Web Server Log Parsing

The Algorithm: You have a massive 50GB text file of web logs. You need to count the total number of times the IP address "192.168.1.5" appears. **The Question:** This requires combining two skeletons. Identify them and describe how they interact simultaneously. **The Answer:**

- **Skeletons:** Map and Reduce (MapReduce).
- **Interaction:**
 1. **Map Phase:** The file is chunked into hundreds of smaller text blocks. The Map skeleton assigns these blocks to workers. Each worker independently scans its text block and returns a local integer count (e.g., "I found the IP 45 times in my chunk").
 2. **Reduce Phase:** The Reduce skeleton takes the array of intermediate local counts returned by the Map phase and performs a parallel tree-reduction using the **Addition** operator to yield the final total count.
- **Code Injection (Python MapReduce):**

```
import multiprocessing as mp

def map_count_ip(log_chunk):
    return log_chunk.count("192.168.1.5")

if __name__ == '__main__':
    chunks = ["chunk1.txt content...", "chunk2.txt content...", "..."]

    with mp.Pool() as pool:
        # Phase 1: MAP (Parallel evaluation of chunks)
        local_counts = pool.map(map_count_ip, chunks)
        # local_counts visually: [45, 12, 0, 89]

        # Phase 2: REDUCE (Sequential here for simplicity, but tree-reduce
        # in true PDC)
        final_total = sum(local_counts)
        print(f"Total occurrences: {final_total}")
```

Scenario 3: Sequence Alignment (Bioinformatics)

The Algorithm: You are comparing two DNA sequences mathematically. To find the optimal alignment score at matrix cell (i,j) , you must already know the scores of $(i-1, j)$, $(i, j-1)$, and $(i-1, j-1)$. **The Question:** Can you use a Divide and Conquer skeleton here? If not, what must you use and why? **The Answer:**

- **Skeleton:** Wavefront.
- **Justification:** Divide and Conquer is inappropriate because it requires splitting a problem into independent sub-problems. In this DNA matrix, the right half of the matrix mathematically depends on the exact results of the left half, preventing independent splitting. We must use the **Wavefront**

skeleton, which processes the matrix along its diagonals. Because cells on the exact same diagonal wave never share dependencies with each other, they can be calculated in parallel safely.

Scenario 4: Banking Transactions

The Algorithm: A bank needs to process 1 million transaction objects (`AccountID`, `Amount`). The goal is to update the final balances of all accounts. **The Question:** Why is the Map skeleton dangerous here? How do you fix it? **The Answer:**

- **Danger:** A Map skeleton executes tasks fully independently. If Transaction 5 (Thread A) and Transaction 100 (Thread B) simultaneously target the exact same `AccountID`, they will trigger a **Race Condition** on the account balance resulting in lost money.
- **Fix:** While Map initiates the parallelization, the inner logic modifying the shared account balances must be protected using explicit synchronization primitives (like an **Atomic Operation** or a `pthread_mutex_t` locked specific to that `AccountID`). Alternatively, group transactions by `AccountID` first, and dispatch arrays of grouped transactions as farm tasks.
- **Code Injection (C Pthreads Fix):**

```
// DANGEROUS MAP LOGIC:
void* process_tx(void* arg) {
    Transaction* tx = (Transaction*)arg;
    accounts[tx->account_id].balance += tx->amount; // RACE CONDITION
}

// FIXED LOGIC:
pthread_mutex_t account_locks[NUM_ACCOUNTS]; // 1 lock per account

void* process_tx_fixed(void* arg) {
    Transaction* tx = (Transaction*)arg;

    // Lock the specific account being modified. Other threads modifying
    // DIFFERENT accounts can proceed!
    pthread_mutex_lock(&account_locks[tx->account_id]);
    accounts[tx->account_id].balance += tx->amount;
    pthread_mutex_unlock(&account_locks[tx->account_id]);
}
```